

Technical Report on the Implemented `main` Corrosion Pipeline

Repository-grounded reconstruction from code, data files, saved artifacts, diagnostics, and local projects

March 17, 2026

Abstract

This report reconstructs the `main` project strictly from its executable Python source, saved models, metric CSVs, cached image-feature table, the current raw workbook under `Data/`, and the thesis/conference PDFs stored in the repository documentation folders. The implemented system is not a single end-to-end image-to-RUL model. It is a three-task scikit-learn pipeline:

1. **current_corrosion**: predict the current label `B_Peak_Rust_Percentage_[%]` from metadata plus handcrafted image features;
2. **progression**: predict that same current corrosion label from metadata plus week only;
3. **time_to_threshold**: predict a derived remaining-weeks target for specimens that eventually cross a fixed 20% peak-rust threshold.

The repository contains strong current-state evidence for visible corrosion quantification and weaker, more assumption-heavy evidence for metadata-driven progression and threshold timing. It also contains substantial reproducibility drift: the current raw workbook no longer matches the identifier and column conventions expected by the code, the image-feature cache is keyed by row-level IDs that the current loader no longer reconstructs, and the deployment manifest stores absolute artifact paths on another machine. This report therefore distinguishes carefully between what the saved artifacts support, what the present code still implements in principle, and what can no longer be rerun from the current raw files without repair.

Contents

1	Problem framing	3
1.1	Actual problem solved by the implemented code	3
1.2	What is directly supervised and what is not	3
1.3	Why direct image-to-RUL is not supported	4
1.4	Exact scope of the implemented system	4
2	Repository-grounded pipeline overview	5
2.1	Key repository components	5
2.2	Pipeline stages	6
2.3	Data ingestion and parsing details	6
2.4	Image alignment and ID mapping	7
2.5	Cleaning and missing/corrupted-image handling	7
2.6	Feature extraction	8
2.7	Feature-family organization	8
2.8	Model training, evaluation, and proxy-RUL logic	8

3	Data description and data dictionary	9
3.1	Current raw data versus saved artifacts	9
3.2	Code-workbook mismatches that matter for reproducibility	9
3.3	Metadata dictionary	9
3.4	Labels, targets, and derived targets	11
3.5	Dense versus sparse labels	13
4	Feature engineering	14
4.1	Implemented image-derived features	14
4.2	What is not implemented	14
4.3	Feature families used by each model	15
4.4	Feature dominance and tautology risk	15
4.5	Redundancy, instability, and campaign dependence	16
5	Technical challenges and implemented responses	16
6	Modeling methodology	17
6.1	Shared preprocessing and evaluation logic	17
6.2	Task A: current corrosion	18
6.3	Task B: progression	18
6.4	Task C: time to threshold	18
6.5	Selected models and saved performance	21
6.6	Model-selection implications	22
7	Split strategies and leakage control	22
7.1	Why random row-wise splitting is invalid	22
7.2	How grouped splitting is implemented	22
7.3	What the implemented split supports	22
7.4	What was not implemented	22
8	Results and interpretation	23
8.1	Aggregate benchmark behavior	23
8.2	Strongly supported results	23
8.3	Weak, negative, and exploratory results	23
8.4	What the results do <i>not</i> justify claiming	24
9	What the model can and cannot learn	24
9.1	What is directly visible and therefore learnable	24
9.2	What may be indirectly inferable	25
9.3	What is hidden and not reliably recoverable here	25
9.4	Why some targets are easier than others	25
9.5	Why metadata may outperform image features in some tasks	25
10	Traceability to original thesis and documentation	26
10.1	What the documentation says	26
10.2	What main actually reproduces	26
10.3	What could not be recovered exactly	26
11	Limitations	27
12	Future work	27
A	Meeting-defense questions and concise evidence-based answers	28

1 Problem framing

1.1 Actual problem solved by the implemented code

The `main` repository implements a multi-model corrosion analytics workflow rather than a single direct predictor. The training script `main/train_models.py` fits three separate supervised regression tasks:

- **Task A: current corrosion.** Input is metadata plus handcrafted image features, and the target is the current observation's `B_Peak_Rust_Percentage_[%]`.
- **Task B: progression.** Input is metadata plus `week`; the target is again the current observation's `B_Peak_Rust_Percentage_[%]`. This is a metadata-conditioned regression of current corrosion against time, not a sequence model.
- **Task C: time to threshold.** Input is metadata, `week`, and `B_Peak_Rust_Percentage_[%]`; the target is a derived value called `remaining_weeks_to_threshold`, computed only for specimens that eventually cross a fixed threshold.

These three tasks share the same repository, but they are not equivalent scientifically. Task A is the closest to an image-based corrosion estimator. Task B is metadata-driven. Task C is a proxy timing model conditioned on current corrosion, not direct image-to-RUL learning.

1.2 What is directly supervised and what is not

The direct supervised quantities in `main` are:

- `B_Peak_Rust_Percentage_[%]` for Task A;
- `B_Peak_Rust_Percentage_[%]` again for Task B;
- `remaining_weeks_to_threshold` for Task C, but this target is derived inside the training script from the corrosion label and `week`, not read as a native raw column.

The following quantities are present in the current workbook but are not trained as targets in `main`:

- `A_Surface_Total_Rust_Percentage[%]`
- `A_Total_Rust_Category_(1-4)`
- `B_Peak_Rust_Category_(1-4)`
- `B_Location_of_Peak_Rus_ in_length_[cm]`
- `Last_Wire_Area_Loss_(Faliure_Surface)_%`
- `Ultimate_Load_[kN]`

The structural labels are especially sparse in the current workbook: both wire-loss and ultimate-load values are non-null in only 48 rows, which corresponds to one terminal measurement per specimen.

1.3 Why direct image-to-RUL is not supported

Direct image-to-RUL is not implemented in this repository for four code-grounded reasons:

1. No RUL column exists in the raw workbook or in the model interface.
2. The only timing target is `remaining_weeks_to_threshold`, which is created inside `train_models.py` by finding the first week where `B_Peak_Rust_Percentage_[%]` exceeds a fixed 20% threshold.
3. The time model is trained on *true current corrosion labels*, not on image-derived predictions.
4. The API also offers a second threshold-related mechanism: the progression endpoint sweeps week through the progression regressor and reports the first predicted crossing week. This is a heuristic threshold-crossing calculation, not an explicit survival or RUL model.

1.4 Exact scope of the implemented system

The executable scope of `main` is:

- parse the raw workbook into a feature table;
- derive specimen and time metadata from identifiers;
- compute or load handcrafted image features using fixed RGB thresholds and global color statistics;
- benchmark several tabular regressors under grouped specimen splits;
- save the best selected pipeline for each task;
- expose current corrosion, progression curve, and time-to-threshold inference through FastAPI.

It does *not* implement:

- direct sequence modeling from image streams;
- structural damage prediction from images;
- leave-one-treatment-out or leave-one-campaign-out evaluation;
- a single end-to-end image-to-threshold or image-to-RUL benchmark;
- monotonicity-constrained progression learning;
- censoring-aware survival analysis.

2 Repository-grounded pipeline overview

2.1 Key repository components

Component	Repository-grounded role
<code>main/data_utils.py</code>	Raw workbook loading, numeric coercion, identifier parsing, derived temporal metadata, image-path construction, and dataset summary.
<code>main/image_features.py</code>	Handcrafted image feature extraction using RGB threshold masks, RGB/HSV statistics, red-dominance features, and cache handling.
<code>main/modeling.py</code>	Candidate model definitions, preprocessing, grouped cross-validation, grouped holdout benchmarking, selection logic, and feature-importance extraction.
<code>main/train_models.py</code>	Orchestrates all three tasks, saves metrics and feature importance tables, dumps the selected models, and writes the artifact manifest.
<code>main/inference.py</code>	Loads the saved manifest and model objects, assembles prediction rows, clips outputs, and implements progression and threshold inference logic.
<code>main/api.py</code>	FastAPI wrapper exposing <code>/health</code> , <code>/predict/current</code> , <code>/predict/progression</code> , and <code>/predict/time-to-threshold</code> .
<code>main/reports/*.csv</code>	Saved benchmark outputs: task-level metrics, feature-importance tables, and the cached image feature matrix.
<code>main/artifacts/*.joblib</code> and <code>manifest.json</code>	Saved production pipelines and metadata from the successful historical training run.

Table 1: Executable repository components used to reconstruct the implemented system.

2.2 Pipeline stages

Stage	Input	Output	Purpose, design choice, and limitation
Data ingestion	Excel workbook + image directory	Raw dataframe with promoted semantic headers	<code>load_corrosion_dataframe()</code> assumes the first data row contains the semantic column names. This reflects the historical data snapshot but is brittle to workbook changes.
Dataset parsing	Raw dataframe	Derived <code>specimen</code> , <code>capture_date</code> , <code>week</code> , <code>series</code> , <code>image_path</code> , <code>image_exists</code>	Designed to reconstruct row-level IDs and align each label row to an image. In the current raw workbook, this assumption is broken because ID is now specimen-level rather than row-level.
Image alignment / pre-processing	Existing PNG files	RGB arrays or cached features	No geometric alignment or segmentation pipeline is executed here. The code assumes preprocessed PNGs already exist and extracts global statistics and simple threshold masks only.
Image feature extraction	One image per row	18 image features + corruption metadata	Uses fixed rust/black/gray RGB thresholds plus RGB/HSV summary statistics. Corrupted images are imputed by column median.
Feature-table assembly	Parsed dataframe + image feature table	<code>full_df</code>	Merges on ID. This worked in the saved run but fails against the current workbook schema because cached image features are keyed by row-level IDs.
Task A benchmark	Metadata + image features	Current corrosion benchmark table and best pipeline	Predict current peak rust using grouped splitting by specimen.
Task B benchmark	Metadata + week	Progression benchmark table and best pipeline	Predict current peak rust as a function of metadata and week. This is not a sequence model and has no monotonicity constraint.
Task C proxy-threshold benchmark	Subset of specimens that cross 20% peak rust	Time-to-threshold benchmark table and best pipeline	Derives a proxy timing target from the corrosion label. Excludes censored specimens entirely.
Artifact and report saving	Benchmarks + models	CSV metrics, feature importance tables, joblib pipelines, manifest	Saves aggregate outputs only. No per-row holdout predictions or specimen-level residual files are stored.
Inference API	Metadata, image path, optional current corrosion input	Current corrosion, progression curve, or threshold timing response	Provides deployment wrappers, but the current manifest stores absolute paths on another machine, so loading currently fails locally.

Table 2: Step-by-step reconstruction of the implemented pipeline and its main risks.

2.3 Data ingestion and parsing details

What the code expects. `main/data_utils.py` expects:

- row-level observation identifiers in the column named ID, formatted as `specimen-date-weekW`;
- columns named `Ageing_Days` and `NaCl%`;
- image files whose names are exactly `ID + ".png"`.

What the current workbook contains. The current workbook under `Data/Images_Dataset_A-Z.xlsx` has two leading semantic rows. After promoting the second row into headers, the available raw columns are `Sample Name`, `ID`, `N_Steel_Mesh`, `Treatment`, `Label_Treatment`, `Ageing Days`, corrosion labels, cover, wire loss, and ultimate load. In this snapshot:

- `Sample Name` holds row-level IDs such as `D01-20240110-0W`;
- `ID` holds specimen IDs such as `D01`;
- `Ageing Days` is spelled with a space, not an underscore;
- no direct raw column named `NaCl%` is exposed.

Consequence. The historical training outputs are real, but the current code no longer parses the current raw workbook correctly. The parser now derives `week`, `specimen`, and `series` as all-missing, creates nonexistent image paths such as `D01.png`, and fails in `dataset_summary()` before training proceeds.

2.4 Image alignment and ID mapping

No explicit alignment, registration, cropping, white-balance correction, or strip segmentation is executed in the code of `main`. Those operations are documented in the thesis and conference paper, but in this repository version they are treated as upstream preprocessing assumptions embodied in the existing PNG files and in the hand-coded RGB thresholds.

The only direct image-to-row mapping implemented in code is filename construction from ID. Because the current raw workbook now stores specimen IDs in that column, the live code maps rows to nonexistent files such as `Data/Images_dataset/D01.png`. In the saved run, the row-level IDs still matched the image filenames, as evidenced by the 792-row image-feature cache.

2.5 Cleaning and missing/corrupted-image handling

Numeric columns are coerced with `pd.to_numeric(errors="coerce")`. The scikit-learn preprocessors then apply median imputation to numeric features and most-frequent imputation to categorical ones.

Image corruption is handled inside `extract_image_features()`. If Pillow fails to open an image, every image feature is returned as `NaN`, the row is flagged as corrupted, and the full feature table later fills the numeric image columns with feature-wise medians.

In the saved artifacts there is exactly one corrupted image:

- `E01-20240508-17W`

The current raw workbook contains 791 labeled rows, while `image_features_cache.csv` contains 792 IDs. The only cached ID not present in the current workbook is again `E01-20240508-17W`, so the corrupted row also coincides with the current data/artifact mismatch.

2.6 Feature extraction

The image branch is entirely handcrafted. `main/image_features.py` creates:

- RGB channel means: `img_r_mean`, `img_g_mean`, `img_b_mean`;
- RGB channel standard deviations: `img_r_std`, `img_g_std`, `img_b_std`;
- HSV channel means and standard deviations;
- `img_rg_diff_mean`;
- `img_red_dominance_pct`;
- `img_rust_mask_pct`, `img_black_mask_pct`, `img_gray_mask_pct`;
- `img_rust_to_black_ratio`.

The thresholds match the documented RGB ranges from the thesis and conference paper:

- rust: [25, 0, 0] to [255, 100, 80];
- black pores: [0, 0, 0] to [5, 5, 0];
- gray/background: [60, 70, 35] to [120, 105, 95].

2.7 Feature-family organization

The model families used in training are:

- metadata/context features: `N_Steel_Mesh`, `Treatment`, `NaCl%`, `Ageing_Days`, `Cover_(Faliure_Surface)_[m]`, `week`, `series`;
- handcrafted image features: 18 global color and threshold-derived image variables;
- the derived threshold-timing input `B_Peak_Rust_Percentage_[%]` for Task C.

There are no deep image embeddings, no morphology descriptors, no strip-wise image features, and no learned image backbone in this repository.

2.8 Model training, evaluation, and proxy-RUL logic

`main/modeling.py` benchmarks fixed candidate regressors under grouped splitting:

- ElasticNet with $\alpha = 0.02$, `l1_ratio = 0.2`, and `max_iter=10000`;
- RandomForestRegressor with 500 trees and `min_samples_leaf=2`;
- GradientBoostingRegressor with default scikit-learn hyperparameters;
- optional XGBoost with 600 estimators, learning rate 0.03, depth 6, subsample 0.9, and `colsample_bytree=0.8`.

Selection is heuristic, not tuned: the code sorts the benchmark table by `holdout_mae`, then `cv_mae`, then `holdout_rmse`. The chosen model is then refit on the full dataset for production.

Proxy threshold logic appears in two places:

1. Task C constructs `remaining_weeks_to_threshold` from the first week a specimen reaches the threshold.
2. The progression endpoint sweeps week through the progression regressor and returns the first predicted crossing week.

These are repository-grounded threshold proxies, not direct survival or RUL models.

3 Data description and data dictionary

3.1 Current raw data versus saved artifacts

Source	Rows	Specimens	Repository-grounded note
Current workbook after both semantic rows	791	48	The current raw workbook exposes row-level identifiers in Sample Name , not in ID .
Current image directory	792 PNGs	48	Includes the corrupted image E01-20240508-17W.png .
Saved image-feature cache	792 cached IDs	48	One cached row has no counterpart in the current workbook: E01-20240508-17W .
Saved manifest and metric CSVs	792 training rows reported	48	Evidence that the historical training run used a data snapshot with 792 aligned rows.

Table 3: The current raw files and the saved training artifacts no longer describe exactly the same dataset snapshot.

3.2 Code-workbook mismatches that matter for reproducibility

Code expectation	Current raw workbook reality	Consequence
ID contains row-level strings like D01-20240110-0W	ID now contains specimen IDs like D01; row-level IDs are in Sample Name	Parsed specimen , week , series , and image_path are wrong in live code.
Ageing_Days exists	Current workbook column is Ageing Days	Feature selection by code no longer matches raw columns.
NaCl% exists as a raw column	No direct NaCl% column is exposed in the current workbook	The raw provenance of this trained feature cannot be reconstructed from the current workbook alone.
B_Peak_Rust_Category_(1-5) constant in code	Current workbook column is B_Peak_Rust_Category_(1-4)	Minor schema mismatch; the category constant is unused, but it signals repository drift.
Image files named from ID	Live code now constructs nonexistent files such as D01.png	image_exists is false for all 791 current rows.
Image-feature cache merges on ID	Cache is keyed by row-level IDs, but the live dataframe now contains specimen IDs	Cached image features no longer align and become all- NaN when merged in the current repository state.

Table 4: Current code-data mismatches that prevent exact reruns from the present raw snapshot.

3.3 Metadata dictionary

Variable	Meaning / unit	Source	Role	Repository-grounded note
Sample Name	Row-level observation ID	Current work-book	Original identifier	Holds strings of the form <code>specimen-date-weekW</code> . Not used by live code, though it now carries the correct row key.
ID	Specimen identifier	Current work-book	Original metadata	Current live code treats this as if it were still the row-level ID.
specimen	Specimen/material identifier	Derived in code	Grouping variable	In the live code this is currently all-missing because the regex is applied to specimen IDs rather than row IDs.
capture_date	Observation date	Derived in code	Temporal metadata	Derived only when the row-level ID is parsed correctly.
week	Observation week number	Derived in code / recoverable from Sample Name	Feature and grouping context	In the current work-book, exact <code>week</code> can still be recovered from <code>Sample Name</code> .
Ageing Days	Exposure duration in days	Current work-book	Original metadata	In the current work-book it satisfies <code>Ageing Days = 7 × week</code> exactly.
Ageing_Days	Exposure duration in days	Expected by code and manifest	Intended feature	Feature name used during historical training; currently absent under that exact raw column name.
Treatment	Treatment code	Current work-book	Metadata / categorical feature	Values in current raw data are {MI, NO, PA, SA, SA_PA, SA_VF, VF}.
Label_Treatment	Additional treatment-family label	Current work-book	Metadata, unused	Values are {D, G, MI, SA, VF}; exact semantic mapping is not documented in the code.

Variable	Meaning / unit	Source	Role	Repository-grounded note
N_Steel_Mesh	Number of mesh layers	Current workbook	Metadata / numeric feature	Used in all three trained tasks.
Cover_(Failure_Surface_Overl)	[mm] thickness on failure surface, mm	Current workbook	Metadata / numeric feature	Used in all three tasks and highly important in the progression model.
NaCl%	Salt concentration	Expected by code and manifest	Intended numeric feature	Present in historical training features but not directly exposed in the current workbook; provenance currently ambiguous.
series	Alphabetic family from specimen prefix	Derived in code	Metadata / categorical feature	Serves as a family or campaign surrogate. The code contains no explicit raw campaign column.
image_path	Path to PNG file	Derived in code	Alignment field	Correct only when row-level IDs are available in the parsed identifier column.
image_exists	Whether the PNG path exists	Derived in code	Data-quality flag	In the current live parse it is false for all rows because the filenames are wrong.

Table 5: Metadata and identifier fields relevant to the `main` pipeline.

3.4 Labels, targets, and derived targets

Variable	Meaning / unit	Density	Role	Repository-grounded note
A_Surface_Total_Rust_Percentage_([%])	Total surface rust percentage	791/791	Original corrosion label, unused target	Dense label. In the current workbook it correlates with B_Peak_Rust_Percentage_[%] at 0.9495.
A_Total_Rust_Category_(\$4)	Surface-rust category	791/791	Unused label	Present but not modeled.

Variable	Meaning unit	/	Density	Role	Repository- grounded note
B_Peak_Rust_Percentage_1%	Peak local rust percentage		791/791 current raw; 792/792 saved artifacts	Primary direct target	Used as the direct regression target in Tasks A and B, and as an input to Task C.
B_Peak_Rust_Category_(1-4)	Peak-rust cate- gory		791/791	Unused la- bel	Current workbook uses a 1–4 coding, while a 1–5 constant exists unused in code.
B_Location_of_Peak_Rus- in_length_[cm]	Location of the peak rust along the specimen, cm		791/791	Unused la- bel	Not used in training or inference.
Last_Wire_Area_Loss_(Failure_Surface)_%	Failure surface area loss		48/791	Structural endpoint, unused	One non-null value per specimen only. Values lie in [0, 0.5698] despite the percent suffix, so the stored unit may be fraction-like rather than percentage points.
Ultimate_Load_[kN]	Terminal ul- timate load, kN		48/791	Structural endpoint, unused	One non-null value per specimen only. Range in the current workbook is 1.60–2.87 kN.
remaining_weeks_to_threshold	Weeks remain- ing until first 20% peak-rust crossing		Derived inside train_model	Proxy timing target	Created only for specimens that actually cross the threshold. Not available as a raw label.

Table 6: Label fields in the raw workbook and their actual role in the implemented system.

3.5 Dense versus sparse labels

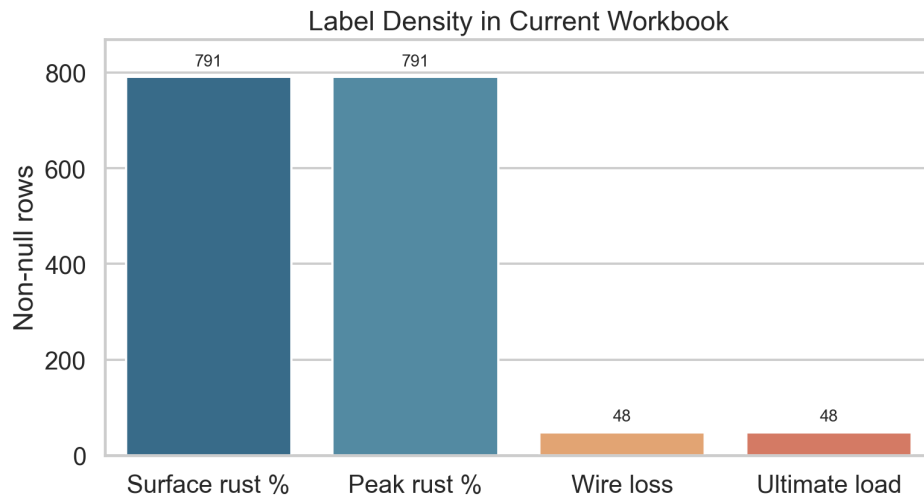


Figure 1: Non-null label counts in the current workbook. Surface and peak rust are dense row-level labels; wire loss and ultimate load are sparse terminal labels, available once per specimen.

Figure 1 shows why direct structural-damage modeling is not supported in `main`: the corrosion labels are dense, but the structural labels are terminal and sparse.

4 Feature engineering

4.1 Implemented image-derived features

Feature family	Implemented variables	Physical meaning and likely correlation	Weaknesses and risks
Global RGB statistics	<code>img_r/g/b_mean</code> , <code>img_r/g/b_std</code>	Capture average channel brightness and color spread; may respond to rust staining, lighting, and surface background.	Sensitive to illumination and preprocessing drift; not localized; can absorb acquisition bias.
Global HSV statistics	<code>img_h/s/v_mean</code> , <code>img_h/s/v_std</code>	Approximate hue, saturation, and brightness distributions; rust tends to alter hue and saturation.	Also global and non-localized; sensitive to white-balance mismatch.
Relative redness	<code>img_rg_diff_mean</code> , <code>img_red_dominance_pct</code>	Measures how strongly red channel exceeds green; likely associated with rust-colored staining.	Strong shortcut risk when the target itself is defined from rust-colored surface evidence.
Threshold masks	<code>img_rust_mask_pct</code> , <code>img_black_mask_pct</code> , <code>img_gray_mask_pct</code>	Estimates global fractions of rust-like, pore-like, and gray/background-like pixels using thesis-matched RGB thresholds.	High tautology risk: the feature engineering reuses a threshold logic close to the documented label-generation pipeline. Still global, not strip-based.
Normalized ratio	<code>img_rust_to_black_ratio</code>	Rust-like area relative to pore-like area. Intended to normalize rust evidence by dark background/void prevalence.	Unstable when black-mask proportion is tiny; denominator protected only by 10^{-6} .

Table 7: Actual image-derived features implemented in `main`.

4.2 What is not implemented

The `main` repository does *not* implement:

- strip-wise corrosion features;
- morphology descriptors;
- connected-component counts;
- texture features such as GLCM or LBP;
- deep CNN embeddings;
- learned segmentation masks;

- brightness normalization beyond whatever is already present in the upstream PNG images.

This matters because the thesis and conference documentation describe strip-based or class-based workflows that are richer than the compact global descriptors used here.

4.3 Feature families used by each model

Task	Feature count	Feature set	Interpretation
Current corrosion	25	7 base metadata features + 18 image features	Mixed model, but the saved feature importance shows it is overwhelmingly driven by the rust-mask feature.
Progression	7	Metadata only: mesh, treatment, intended saline, ageing days, cover, week, series	Purely metadata-driven conditional corrosion regression; not image-driven and not sequence-aware.
Time to threshold	8	Base metadata + B_Peak_Rust_Percentage_[%]	Proxy timing model conditioned heavily on current true corrosion and week.

Table 8: Feature sets used by the three trained tasks.

4.4 Feature dominance and tautology risk

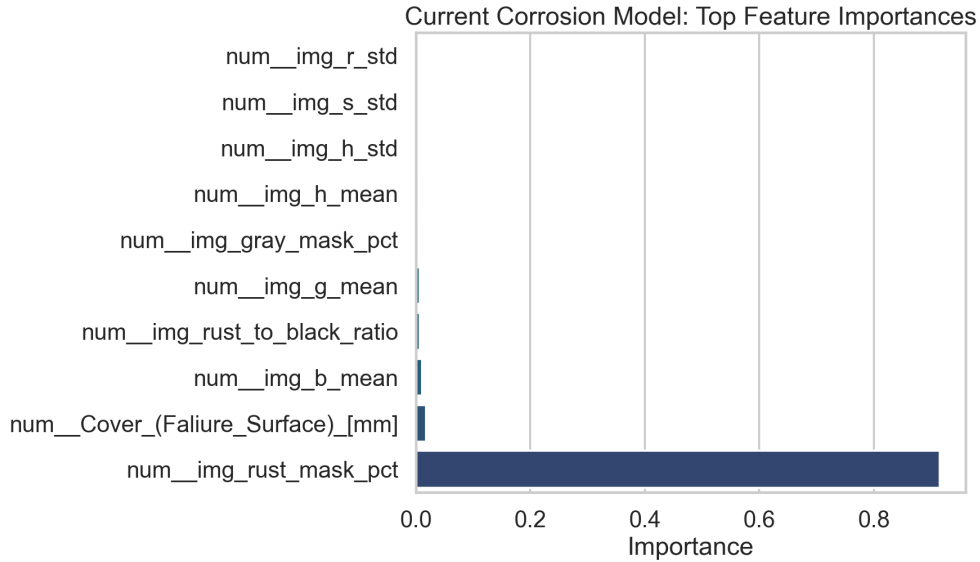


Figure 2: Top feature importances for the selected current-corrosion model. The handcrafted rust-mask percentage dominates the model.

The saved `current_corrosion` feature-importance table shows that `img_rust_mask_pct` alone contributes 0.915 of the total RandomForest importance. The top two features (`img_rust_mask_pct` and cover thickness) sum to 0.933, and the top five sum to 0.958.

This is scientifically important:

- The handcrafted rust-mask percentage uses the same rust RGB threshold family described in the thesis and conference paper.

- When the cached image features are joined back to the current workbook by row-level identifier, `img_rust_mask_pct` correlates with `B_Peak_Rust_Percentage_[%]` at 0.9495.
- The current-corrosion model is therefore highly interpretable, but it is also partly reconstructive: it is learning from a handcrafted visual quantity that is very close to the visible rust label.

4.5 Redundancy, instability, and campaign dependence

Three concrete risks appear in the implemented features:

1. **Temporal duplication.** In the current workbook, `Ageing Days` equals $7 \times \text{week}$ exactly. Yet the code trains on both `Ageing_Days` and `week`, producing perfect temporal redundancy.
2. **Family dependence.** `series`, cover thickness, treatment, and intended saline condition encode specimen-family structure. These are not purely corrosion descriptors.
3. **Inference inconsistency.** The progression and time-to-threshold APIs ask the user for both `week` and `ageing_days`. The README examples hold `ageing_days=260` while varying or using weeks such as 22 or 28, even though the current raw data satisfy `Ageing Days = 7 \times week`. That means the documented API usage is out-of-distribution relative to the current raw data.

5 Technical challenges and implemented responses

Challenge	How it was detected	Implemented handling	Residual risk
Workbook schema drift	Current raw workbook now uses <code>Sample Name</code> , <code>Ageing Days</code> , and specimen-level ID.	None in live code. Historical artifacts still exist.	Fresh training now fails before benchmarking, so the repository is not currently reproducible end to end.
Image-ID misalignment	Live code constructs <code>D01.png</code> -style paths; all 791 current <code>image_exists</code> flags become false.	None in live code. The saved image-feature cache preserves the historical aligned IDs.	Even cached features no longer merge because the current live dataframe uses the wrong IDs.
Corrupted image file	<code>image_features_cache.csv</code> and <code>manifest.json</code> both record <code>E01-20240508-17W</code> as corrupted.	Median-impute the image features.	The corrupted row is also the row missing from the current workbook, so exact raw/artifact alignment is uncertain.
Sparse structural supervision	Current workbook has only 48 non-null wire-loss rows and 48 non-null ultimate-load rows.	No structural model is trained in <code>main</code> .	Any structural or hidden-damage claim would exceed the evidence.

Challenge	How it was detected	Implemented handling	Residual risk
Repeated-measures leakage risk	Each specimen has 15, 17, or 18 observations across time.	Group-aware hold-out split and GroupKFold by specimen.	Leakage by exact specimen is controlled, but no stricter treatment/campaign transfer tests are implemented.
Metadata shortcut learning	Progression and time models rely entirely or primarily on metadata and current corrosion values.	None beyond grouped splitting.	Strong current performance may not imply physically robust transfer.
Perfect temporal redundancy	In the current workbook Ageing Days equals $7 \times$ week exactly.	None; both are still used as separate features in code.	Redundant timing inputs can distort feature-importance interpretation and create inconsistent API usage.
Holdout-based model selection	<code>evaluate_models()</code> sorts by holdout MAE, then refits the selected model on the full dataset.	None.	The chosen production model is selected using the same holdout metrics later reported, and the final saved artifact is not the exact pipeline used to produce those holdout scores.
Path portability	<code>manifest.json</code> stores absolute paths under <code>/Users/youseffayaz/...</code>	None.	<code>load_artifacts()</code> fails locally even though the local joblib files exist.
Documentation mismatch	README and <code>scientific_solution.txt</code> imply deployable status and show temporally inconsistent examples.	None.	User-facing documentation overstates current reproducibility and blurs the distinction between proxy timing and direct RUL.

Table 9: Real implementation challenges and their remaining risks.

6 Modeling methodology

6.1 Shared preprocessing and evaluation logic

All three tasks use the same general scikit-learn design:

- numeric features: median imputation + standardization;
- categorical features: most-frequent imputation + one-hot encoding with unknown-category tolerance;
- a single grouped holdout split generated by `GroupShuffleSplit(test_size=0.2, random_state=42)`;

- grouped cross-validation on the training portion using `GroupKFold` with up to 5 folds;
- candidate models ranked by `holdout_mae`, then `cv_mae`, then `holdout_rmse`.

No grid search, random search, repeated holdout, or uncertainty quantification is implemented.

6.2 Task A: current corrosion

Target. `B_Peak_Rust_Percentage_[%]`.

Feature set. 7 base metadata fields plus 18 handcrafted image features.

Best selected model and interpretation. RandomForest was selected by holdout MAE. The saved holdout MAE is 2.3794, RMSE 4.2543, and $R^2 = 0.8996$. This is the strongest result in the repository, but it must be interpreted as visible corrosion quantification, not hidden-damage inference.

What the model can and cannot learn. Because the dominant signal is `img_rust_mask_pct`, the model can learn current visible rust evidence very well. It cannot directly learn structural damage or remaining life.

6.3 Task B: progression

Target. `B_Peak_Rust_Percentage_[%]` again.

Feature set. Metadata only: mesh count, treatment, intended saline, ageing days, cover, week, and series.

Best selected model and interpretation. GradientBoosting was selected by holdout MAE, with holdout MAE 6.7144, RMSE 9.3294, and $R^2 = 0.5170$. The feature importance table shows that `week` (0.468) and cover thickness (0.356) dominate, with the top five features summing to 0.952. This is best interpreted as a metadata-conditioned corrosion regression across observed experimental conditions. It is not an end-to-end image progression model, and it does not encode temporal dynamics explicitly.

Important limitation. The progression inference endpoint constructs curves by sweeping `week` while leaving the other inputs fixed. The code does not enforce consistency between `week` and `ageing_days`, nor does the estimator enforce monotonicity.

6.4 Task C: time to threshold

Target. `remaining_weeks_to_threshold`, derived in code as the first threshold week minus the current week.

Training population. Only specimens that reach the threshold are included. Using the current workbook and the saved 20% threshold, 22 of 48 specimens reach the threshold and 26 are censored out. The resulting timing dataset contains 278 rows.

Feature set. Base metadata plus the true current `B_Peak_Rust_Percentage_[%]`.

Best selected model and interpretation. RandomForest was selected with holdout MAE 2.2114 weeks, RMSE 3.0947, and $R^2 = 0.7820$. However, the feature-importance table shows that current peak rust alone contributes 0.655 of the importance, with **week** adding 0.189. This is therefore a proxy model that mostly learns from the current corrosion state and the current time index.

Critical scope limitation. These timing metrics are *not* end-to-end image-to-threshold results. The benchmark uses true current corrosion labels as inputs. If a deployed pipeline feeds this model with predicted corrosion instead, that compounded error is not evaluated anywhere in the saved outputs.

6.5 Selected models and saved performance

Task	Selected model	Holdout MAE	Holdout RMSE	Holdout R^2	CV MAE	Repository- grounded in- ter- pre- ta- tion
Current corrosion	RandomForest	2.3794	4.2543	0.8996	3.0204	Strong same- observation visible- rust pre- dic- tion driven mostly by a hand- crafted rust- mask fea- ture.
Progression	GradientBoosting	6.7144	9.3294	0.5170	6.6850	Moderate metadata- driven fit for cur- rent cor- ro- sion across weeks, not an im- age model and not a dy- namic model.
Time to threshold	RandomForest	2.2114 weeks	3.0947 weeks	0.7820	2.6535 weeks	Proxy tim- ing model con- di- tional on true cur- rent cor-

6.6 Model-selection implications

The selection criterion is operationally simple but scientifically important:

- current corrosion: ElasticNet has a slightly better holdout R^2 than RandomForest, but RandomForest is selected because its holdout MAE is smaller;
- time to threshold: XGBoost has a better holdout RMSE and R^2 than RandomForest, but RandomForest is still selected because its holdout MAE is marginally smaller;
- the selected production model is then refit on the full dataset, so the joblib artifact is not the exact fitted object that produced the reported holdout score.

7 Split strategies and leakage control

7.1 Why random row-wise splitting is invalid

The data contain repeated observations of the same specimens over time. A random row-wise split would place near-duplicate corrosion states from a single specimen into both training and evaluation, yielding overly optimistic performance.

7.2 How grouped splitting is implemented

`evaluate_models()` first uses `GroupShuffleSplit` with `test_size=0.2` on the specimen groups to create one holdout split. It then evaluates each candidate model with `GroupKFold` on the training portion only, using up to five folds. This gives:

- one grouped holdout estimate;
- one grouped cross-validation average on the non-holdout portion.

7.3 What the implemented split supports

The implemented split supports only a limited statement: the saved models were benchmarked on held-out specimens drawn from the same overall experimental repository.

It does *not* support claims about:

- leave-one-treatment-out transfer;
- leave-one-campaign-out or leave-one-series-out transfer;
- forward-only temporal generalization;
- external image-preprocessing generalization;
- censored-event generalization for the threshold model.

7.4 What was not implemented

No leave-one-treatment-out, leave-one-series-out, leave-one-campaign-out, repeated random seeds, or time-forward split is saved or implemented in `main`. Therefore any precise statement about performance degradation under stricter regimes would be unsupported here.

8 Results and interpretation

8.1 Aggregate benchmark behavior

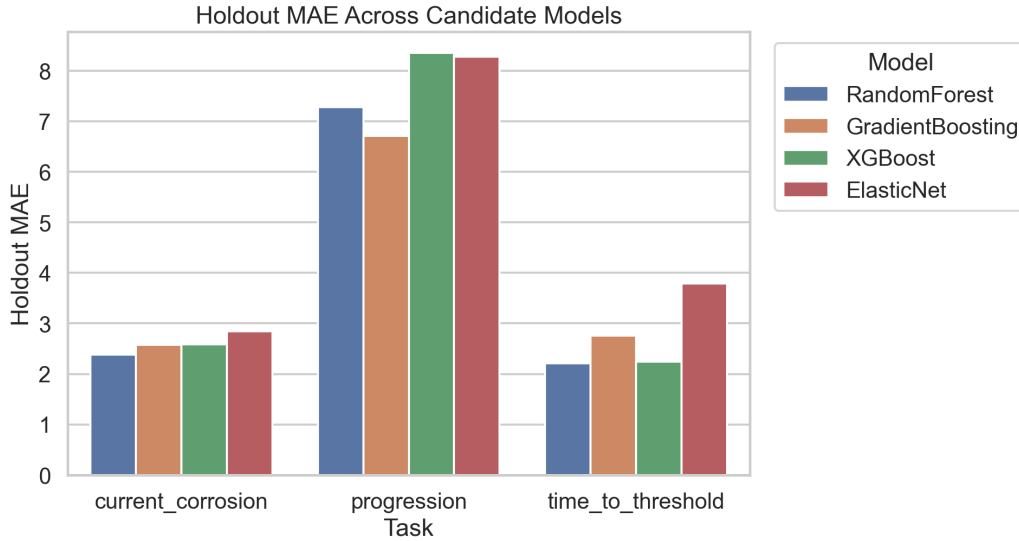


Figure 3: Holdout MAE across candidate models for the three tasks. Current-corrosion prediction is the strongest task; progression is the weakest; time-to-threshold is numerically strong but conditional on a derived timing target and true current corrosion input.

8.2 Strongly supported results

1. The repository supports strong current visible-corrosion estimation. Task A achieves holdout MAE 2.3794 and $R^2 = 0.8996$. The saved importance table and Figure 2 show that this performance is image-driven in a very specific way: it is dominated by the handcrafted rust-mask percentage.

2. The progression model is materially weaker and metadata-driven. Task B degrades to holdout MAE 6.7144 and $R^2 = 0.5170$. The feature-importance table shows that week and cover dominate. This supports a narrower claim: the repository can regress corrosion level from metadata and time within the observed specimen families. It does not support a claim of learned corrosion dynamics from images.

3. The threshold-timing model is a proxy conditioned on current corrosion. Task C achieves holdout MAE 2.2114 weeks and $R^2 = 0.7820$, but this number is conditional on true current corrosion being known and on the specimen belonging to the threshold-reaching subset.

8.3 Weak, negative, and exploratory results

Weak result: Task C is not a complete end-to-end pipeline metric. Because the time model uses the true `B_Peak_Rust_Percentage_[%]` as an input feature during training and evaluation, its holdout score is not the performance of an image-to-threshold system. It is a conditional proxy benchmark.

Negative result: current reproducibility is broken. The training entry point currently fails on the present workbook with a `TypeError` inside `dataset_summary()`, because `week` is all-missing after the identifier parser is applied to the current specimen-level ID column. Even

if that failure were patched, the cached image-feature merge would still return all-NaN image features because the IDs no longer align.

Negative result: deployment claims in the repository are currently overstated. `load_artifacts()` fails locally because `manifest.json` stores model paths under another user's home directory. The joblib files exist locally, but the current deployment code does not resolve them.

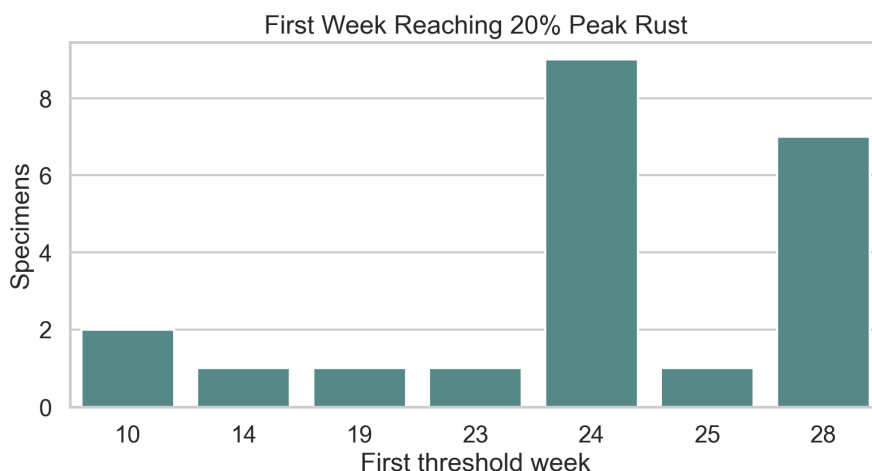


Figure 4: Distribution of the first observed week reaching the 20% peak-rust threshold among the 22 threshold-reaching specimens. The proxy timing dataset is concentrated late in the campaign, especially at weeks 24 and 28.

Exploratory result: threshold-reaching specimens cluster at late weeks. Figure 4 shows that first threshold crossings are concentrated near weeks 24 and 28. This matters because the timing model is trained on a limited event-time distribution rather than on a broad range of failure regimes.

8.4 What the results do *not* justify claiming

The saved outputs do not justify claiming that `main`:

- predicts structural wire loss or ultimate load;
- performs direct image-to-RUL learning;
- has been validated on censored specimens for threshold timing;
- has been validated under treatment- or campaign-held-out transfer;
- reproduces the full strip-based thesis pipeline;
- is currently deployable without repair, despite what some repository prose implies.

9 What the model can and cannot learn

9.1 What is directly visible and therefore learnable

Task A can learn visible surface-rust patterns because it sees:

- the global proportion of pixels within the documented rust RGB range;
- gross channel intensity differences associated with rust staining;
- brightness and saturation changes across the specimen surface.

This is why current-corrosion prediction is the strongest task in the repository.

9.2 What may be indirectly inferable

Through the metadata and family variables, the models may also infer:

- specimen family or campaign-like grouping through **series**;
- treatment condition through **Treatment**;
- exposure stage through **week** and **Ageing_Days**;
- broad material design through cover thickness and mesh count.

These are useful for interpolation within the observed dataset, but they are also potential shortcut variables.

9.3 What is hidden and not reliably recoverable here

The implemented repository does not directly support reliable recovery of:

- hidden internal corrosion or wire area loss from images;
- ultimate load from surface images;
- failure time under censoring;
- true campaign-robust remaining useful life;
- physically constrained monotonic degradation trajectories.

9.4 Why some targets are easier than others

The difficulty ranking in the saved outputs is scientifically coherent:

- **Current corrosion** is easiest because the handcrafted features directly measure visible rust-like pixels.
- **Progression** is harder because the model sees no image and must infer corrosion from metadata plus week.
- **Time to threshold** appears stronger numerically than progression because it is trained on a subset of threshold-reaching specimens and receives the true current corrosion label as an input.

9.5 Why metadata may outperform image features in some tasks

In Task B and Task C, metadata dominate because the task definitions themselves rely on temporal and specimen context. In this repository, metadata do not merely supplement the images; in two of the three tasks they are the entire model input or the majority of the explanatory signal.

10 Traceability to original thesis and documentation

10.1 What the documentation says

The local thesis and conference paper describe a broader workflow with:

- GIMP-based cropping and white-balance preprocessing;
- BIMP batch processing to a standard PNG format;
- MATLAB RGB-threshold segmentation;
- strip-wise analysis along specimen length;
- corrosion categorization into low, medium, and high evidence;
- in the conference paper, a ResNet-50 classifier trained on those categories.

10.2 What main actually reproduces

Documented method	What main reproduces	Expected consequence of the mismatch
GIMP/BIMP preprocessing with white balance, crop, and batch export	Not executed in code; assumed upstream in the existing PNG set	If current PNGs drift from the documented preprocessing, feature behavior can shift.
MATLAB RGB-threshold corrosion extraction	Partially reproduced only through three fixed RGB threshold masks and global color statistics	The code preserves the threshold logic but not the full segmentation workflow.
Strip-wise peak-corrosion computation along specimen length	Not reproduced in feature extraction	The current image features are global, whereas the target label originates from a peak quantity.
Three-class corrosion categorization at $< 1\%$, $1\% - < 20\%$, and $\geq 20\%$	Reused in current-corrosion class labeling and in threshold logic	The threshold semantics are preserved, but the learned task is regression, not classification.
ResNet-50 transfer learning on corrosion categories (conference paper)	Not reproduced in <code>main</code> ; this repository uses classical regressors on handcrafted features	The saved results are not directly comparable to the conference model architecture or accuracy claims.
Scientific-solution text claiming validated local deployment	Not currently true in code because the manifest paths are stale	User-facing documentation overstates present deployability.

Table 11: Comparison between the documented original workflow and the executable `main` implementation.

10.3 What could not be recovered exactly

The repository does not contain the original MATLAB strip-analysis code, the full BIMP configuration, or a documented rule for reconstructing the `NaCl%` feature from the current raw workbook. Therefore the exact preprocessing and feature-assembly path that produced the historical saved models cannot be fully recovered from the current raw files alone.

11 Limitations

The main limitations are specific to the actual repository state:

1. **Three tasks, three different scientific meanings.** The repository is not one coherent image-to-life model. It mixes visible-corrosion regression, metadata-conditioned interpolation, and derived threshold timing.
2. **Sparse structural endpoints.** Wire loss and ultimate load are terminal specimen-level labels only and are not modeled.
3. **Proxy timing target.** The time-to-threshold model excludes 26 censored specimens and uses true current corrosion as an input.
4. **No end-to-end threshold evaluation.** The saved outputs do not measure chained performance from image $\rightarrow$ current corrosion prediction $\rightarrow$ time-to-threshold prediction.
5. **Perfect temporal redundancy.** Ageing Days and week are redundant in the current workbook.
6. **Handcrafted-feature tautology risk.** The dominant current-corrosion feature is a rust-threshold area closely aligned with the documented corrosion labeling logic.
7. **No strict transfer splits.** The repository does not evaluate leave-one-treatment-out, leave-one-series-out, or campaign-held-out robustness.
8. **Aggregate outputs only.** No holdout predictions, residual plots, or saved split memberships are available for deeper error auditing.
9. **Broken current reproducibility.** The live training code no longer matches the present raw workbook, the current cache no longer merges, and the live API loader fails on stale absolute paths.
10. **External validity remains narrow.** The results are tied to one experimental repository, one image-preprocessing lineage, and a handcrafted visual threshold strategy.

12 Future work

The next steps that logically follow from the actual repository state are repair and disambiguation, not generic model escalation:

1. **Repair the data contract.** Update the loader to parse row-level IDs from `Sample Name`, rename or map `Ageing Days` to `Ageing_Days`, and document how `NaCl%` should be reconstructed.
2. **Make artifact loading relative-path based.** The current manifest prevents local deployment even though the model files exist.
3. **Persist split membership and per-row predictions.** Without them, the current results cannot be audited at specimen level.
4. **Decide whether the project is primarily image-based or metadata-based.** In main, Tasks B and C are not image-driven; that should be an explicit design choice rather than an accidental side effect.

5. **If threshold timing is the real goal, evaluate end-to-end chaining.** Measure the full image $\rightarrow$ current corrosion $\rightarrow$ time-to-threshold path rather than only the conditional time model.
6. **If thesis reproducibility matters, restore strip-wise logic.** The current global image features are not the documented strip-analysis pipeline.
7. **If hidden structural degradation matters, design a new supervision strategy.** The current labels are too sparse and too terminal for direct structural modeling.
8. **Remove or constrain redundant temporal inputs.** The progression and timing endpoints should not accept inconsistent `week/ageing_days` combinations.

A Meeting-defense questions and concise evidence-based answers

Why not direct RUL? Because no raw RUL label exists, no survival model is implemented, and the time target in `main` is a derived threshold-crossing proxy created from the corrosion label and week.

Why these split strategies? Because each specimen is observed repeatedly over time. Grouping by specimen is the minimum required control against repeated-measures leakage.

Why did robustness drop from current corrosion to progression? The strongest current model uses image features dominated by rust-mask percentage, which directly measures visible corrosion evidence. The progression model has no image input and relies mostly on week and cover, so it solves a less direct problem.

Why can visible corrosion be predicted better than structural degradation? Visible corrosion is present in the image and in dense labels. Structural damage variables are hidden and only sparsely observed at specimen endpoints.

How much of the signal comes from metadata? It depends on the task. For current corrosion, the dominant saved feature is the image-derived rust-mask percentage. For progression and timing, metadata and current corrosion carry nearly all of the usable signal.

What are the highest-risk assumptions? The largest risks are the broken ID mapping, the missing documented raw `NaCl%` field, the use of holdout MAE for model selection, the assumption that the current PNGs still reflect thesis-style preprocessing, and the use of true current corrosion in the timing benchmark.

Could preprocessing mismatch explain part of the gap between documentation and code? Yes. The thesis and conference documents emphasize GIMP/BIMP preprocessing and strip-based MATLAB analysis, but `main` uses compact global descriptors and assumes the PNG set is already normalized appropriately.

Which features are most interpretable? The most interpretable features are the hand-crafted mask percentages, especially `img_rust_mask_pct`, and the metadata fields such as week, cover, and treatment. These are far more interpretable than latent deep features, but they also make tautology and shortcut risks easier to see.

What exactly is meant by each major output?

- **current corrosion:** current peak-rust percentage for the same observation.
- **progression curve:** a metadata-conditioned sweep of predicted peak rust over week, not a sequence forecast with memory.
- **predicted threshold week from progression:** first week on that swept curve whose predicted rust exceeds the threshold.
- **time to threshold:** a separate regression model's predicted remaining weeks, conditioned on current corrosion and metadata.

Are the two threshold outputs guaranteed to agree? No. The repository contains two separate threshold mechanisms: progression-curve crossing and the dedicated time-to-threshold regressor. No consistency check or calibration between them is saved.

Can the API be used as-is now? No. The current `manifest.json` points to model files on another machine, so `load_artifacts()` fails locally unless the paths are repaired.